Screening verification: 60-record sample

Python for DO-178C-compliant airborne software — systematic literature review

This pack contains 60 records drawn from the screening set: 30 that the automated screening included and 30 that it excluded, in random order. For each one you are shown the title, venue, keywords where available, and abstract, followed by what the screening decided in each round and the reason it gave.

For each record, decide whether you agree with the final decision. Where you agree, move on; nothing needs recording. Where you **disagree**, open `VERIFY_60_response.csv`, find that record number, and put `d` in the `disagree` column with your reason in the `why` column. The other columns are already filled in; leave them alone. Only the disagreements are of interest, and the review reports how many of the 60 were disputed and on what grounds.

The criteria are in `slr/protocol.md` §5. Judge each record against those criteria, not against what you would personally have retrieved.

Not every record went through every round, so the rounds shown differ from page to page. Round 1 screened all 3,709 records. Round 2 re-read only those whose relevance fell in the band where repeated screening proved unreliable. Round 3 rescored only the records that were included after rounds 1 and 2, to put them on one comparable scale. Where a round is absent, that record was not eligible for it; where two rounds disagree, the later one stands.

Records without an abstract are marked as such: 8 of the 60 carry a title only, which is itself a limitation worth noting where it affects your judgement.

RECORD 1 OF 60

Co-evolution of Types and Dependencies: Towards Repository-Level Type Inference for Python Code

2026 · Proc. ACM Softw. Eng.

Keywords. Entity Dependency Graph, Large Language Models, Repository-level, Type Inference

Abstract. Python's dynamic typing mechanism, while promoting flexibility, is a significant source of runtime type errors that plague large-scale software, which inspires the automatic type inference techniques. Existing type inference tools have achieved advances in type inference within isolated code snippets. However, repository-level type inference remains a significant challenge, primarily due to the complex inter-procedural dependencies that are difficult to model and resolve. To fill this gap, we present PyTIR, a novel approach based on LLMs that achieves repository-level type inference through the co-evolution of types and dependencies. PyTIR constructs an Entity Dependency Graph (EDG) to model the objects and type dependencies across the repository. During the inference process, it iteratively refines types and dependencies in EDG for accurate type inference. Our key innovations are: (1) an EDG model designed to capture repository-level type dependencies; (2) an iterative type inference approach where types and dependencies co-evolve in each iteration; and (3) a type-checker-in-the-loop strategy that validates and corrects inferences on-the-fly, thereby reducing error propagation. When evaluated on 12 complex Python repositories, PyTIR significantly outperformed prior works, achieving a TypeSim score of 0.89 and a TypeExact score of 0.84, representing a 27% and 40% relative improvement over the strongest baseline. More importantly, PyTIR removed new type errors introduced by the tool by 92.7%. This demonstrates a significant leap towards automated, reliable type annotation for real-world Python development.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I4, relevance 5/5

The paper presents and evaluates a Python type-inference technique validated by a type checker, directly informing the static/type-check stage of RQ2.

Round 3 — rescoring of the included set

include, criterion I3, relevance 4/5

It would populate the Python tool/approach and evaluation-scale fields, and offers contextual evidence for RQ2 on type inference paired with checking. It does not establish verification soundness or DO-178C relevance.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 2 OF 60

Use of the RTCA DO-330 in aeronautical databases

2015 · 2015 IEEE/AIAA 34th Digital Avionics Systems Conference (DASC)

Keywords. Databases;Qualifications;Software;Safety;Standards organizations;Organizations

Abstract. Standards in safety-critical software require tool qualification. On December 2011, the DO-330, a dedicated standard for tool qualification to be used in airborne and CNS/ATM systems was released and provided guidance to conduct the Tool Qualification Process. After that, with some adaptations, other domains or technologies such as automotive, space, systems, electronic hardware, aeronautical databases, and safety assessment processes were able to use the DO-330. This paper describes a successful case of use and adaptation of this standard to Aeronautical Databases, within the scope of the new RTCA DO-200B.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I5, relevance 3/5

The case provides relevant background on applying DO-330 tool qualification in an aeronautical context, though it does not address Python or directly establish which DO-178C objectives a Python toolchain can support.

Round 2 — re-read of borderline records

include, criterion I5, relevance 4/5

A full reading could provide evidence about applying DO-330 tool-qualification guidance in an aeronautical certification context, including the approach and evidence used. This bears on the extraction field for relevance to DO-178C/DO-330 objectives and practices.

Round 3 — rescoring of the included set

include, criterion I5, relevance 4/5

A full reading could provide DO-330 tool-qualification evidence, including the adaptation and process used in an aeronautical context, relevant to RQ3. It is not Python-specific, but it is a direct certification precedent.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 3 OF 60

C-17 production automated verification equipment (PAVE)

1998 · 1998 IEEE AUTOTESTCON Proceedings. IEEE Systems Readiness Technology Conference. Test Technology for the 21st Century (Cat. No.98CH36179)

Keywords. Automatic testing;Aircraft;System testing;Aerospace electronics;Automatic control;Cities and towns;Aerospace control;Production facilities;Assembly;Production systems

Abstract. Manually testing aircraft functions during factory assembly has been a costly and time-consuming task. In addition, this problem has been compounded by the fact that deliverable cockpit avionics are required during testing and are therefore exposed to the damaging production environment longer. The C-17 aircraft currently requires 48 on-aircraft test procedures (OATP) to verify system operations and airworthiness prior to customer delivery. As a result, the current manual test method equates to over 15,000 test hours per aircraft. This paper discusses the implementation of the Production Automated Verification Equipment (PAVE). PAVE is an automated aircraft test system which, by emulating various cockpit LRUs and test procedures, automates the current manually intensive OATP testing.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 2/5

The paper concerns production testing of C-17 aircraft systems, but does not address language analysis, formal verification, or which DO-178C objectives are attainable or how evidence for them is produced.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 4 OF 60

RSM: A Real-time Security Monitoring Platform for IoT Networks

2023 · 2023 IEEE Canadian Conference on Electrical and Computer Engineering (CCECE)

Keywords. IoT security;Deep Learning;CNN;LSTM;DNN;IoT23 dataset;Power BI dashboard;Test bed;Raspberry PI;Real-time prediction; Deep learning;Visualization;Computational modeling;Predictive models;Real-time systems;Security;Internet of Things

Abstract. The rapid growth of Internet of Things (IoT) resulted in a heightened risk of security breaches, as cybercriminals have begun to target IoT devices and networks with increasingly sophisticated techniques. However, IoT security monitoring platforms face several challenges, including the inability to identify unknown threats, limited real-time prediction capabilities depending on signature-based threat identification, and the need for standardization and integration issues. In this paper, we propose a Real-Time Security Monitoring (RSM) platform based on the results of Deep Learning models, which can predict attacks on IoT networks and visualize the prediction results in a custom-built Power BI dashboard in a real-time manner. To evaluate our proposed solutions, we compare the effectiveness of three deep learning models - Convolutional Neural Networks (CNN), Long Short-Term Memory (LSTM), and Deep Neural Networks (DNN) - using the IoT23 dataset in the context of the binary classification problem. We compare these models based on their accuracy, precision, recall, and F1 score. In addition, our findings show that our proposed platform outperforms existing solutions in terms of accuracy and can predict IoT network attacks with high precision and recall. We also implemented a test bed using a Raspberry PI programmed to send its logs to the nearest connected edge router and a server programmed using Python with a scheduler to pull those logs and show real-time Deep Learning Model prediction results in a Power BI dashboard. Our results demonstrate that the RSM and the Power BI dashboard provide a user-friendly way to monitor IoT Network security in real-time. This study provides valuable insights into applying Deep Learning (DL) and Power BI dashboard in the IoT security monitoring domain.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 1/5

Python appears only as an implementation detail in an IoT security-monitoring study, with no relevant safety, verification, language-restriction, or toolchain-feasibility content.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 5 OF 60

Qualification of a model checker for avionics software verification

2017 · Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)

No abstract available — this record was judged on its title alone.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I5, relevance 3/5

The title indicates work on qualifying a verification tool for avionics, which is relevant to how verification evidence is produced, though no Python or specific DO-178C objective is identified.

Round 2 — re-read of borderline records

include, criterion I5, relevance 4/5

The title indicates evidence about qualifying a verification tool for avionics software, which could populate the tool/framework, verification approach, and DO-330/DO-178C relevance fields.

Round 3 — rescoring of the included set

include, criterion I5, relevance 4/5

A full reading could provide evidence about DO-330 tool qualification and how verification evidence for avionics software is produced, filling the tool-qualification and DO-178C-relevance fields.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 6 OF 60

Advances on Real Time M/EEG Neural Feature Extraction

2025 · 2025 IEEE 38th International Symposium on Computer-Based Medical Systems (CBMS)

Keywords. MlEEG;Neurofeedback;Neural feature; Electric potential;Power measurement;Feature extraction;Real-time systems;Electroencephalography;Entropy;Brain-computer interfaces;Frequency measurement;Recording;Neurofeedback

Abstract. This paper introduces MNE-RT, a Python package designed for real-time neural feature extraction from magne-toencephalography (MEG) and electroencephalography (EEG) signals in Brain-Computer Interface (BCI) systems. The package incorporates efficient algorithms spanning traditional univariate metrics, such as frequency band power and entropy, to advanced bivariate connectivity measures. It is compatible with various recording systems, enabling the extraction of neural targets from brain signals in real time, with potential applications in enhancing neurofeedback efficacy.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 1/5

The paper concerns neural-signal feature extraction, not Python runtime timing determinism, verification, language restriction, or safety-oriented toolchain feasibility.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 7 OF 60

Real-Time Detection and Identification of ARP Spoofing Attacks in Microgrids

2025 · IEEE Access

Keywords. ARP spoofing;false data injection;man-in-the-middle;microgrids;network intrusion detection system;cyber-physical power systems; Protocols;Real-time systems;Frequency control;Computer crime;Power systems;Voltage control;Microgrids;Synthetic aperture sonar;IP networks;Media Access Control

Abstract. The integration of smart devices and advanced communication infrastructure has turned power systems into cyber-physical systems (CPS), introducing cyber vulnerabilities. One such vulnerability arises from the use of the address resolution protocol (ARP), which is commonly employed in power systems' information technology (IT) infrastructure to assign internet protocol (IP) addresses to devices such as relays, controllers, and meters. Due to the lack of authentication in ARP, attackers can exploit it to infiltrate substation automation systems (SAS). To detect and locate ARP spoofing attacks, a novel network intrusion detection system (NIDS) was developed using Snort3, TShark, and Python scripts to monitor ARP broadcast messages. This detection method was tested on a dedicated, real-time multi-agent CPS testbed, where a microgrid is simulated as the physical layer using a real-time digital simulator (RTDS), while the cyber layer consists of a multi-agent control implemented in a graphical network simulator (GNS3) together with Raspberry Pi devices. The real-time operator's view is developed in Grafana visualization, mimicking the real-world microgrid operation. Two common practical ARP attacks, known as man-in-the-middle (MITM) and false data injection (FDI) attacks, were conducted to evaluate the performance of the proposed NIDS method. Both MITM and FDI attacks were implemented using IT network testing tools, such as Ettercap and the Scapy library in Python. The results have shown that the proposed NIDS system can detect, localize, and publish the IP address of the attacker in both MITM and FDI attack scenarios. In addition, the impact analysis results indicated that for an identical malicious payload, the FDI attack is more severe when compared to MITM due to the intermittent nature of false data injection.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E3, relevance 1/5

Python is used to implement a network intrusion detector, but the record does not address Python or comparable-language behavior, verification, language restriction, or runtime timing predictability relevant to the review.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 8 OF 60

VDLA:Vehicle Identity Fraud Using an IoT-Enabled License Verification and Biometric Authentication

2026 · 2026 International Conference on Electrical/Electronics, Robotics, Artificial Intelligence, and Informatics (ICERAI)

Keywords. Cybersecurity;Vehicle security system;Multi-factor authentication;License verification;Fingerprint authentication;Mobile application; Breadboard;Circuits;Electronic circuits;Microcontrollers;Microprocessors;Circuits and systems;Internet of Things;Near field communication;Communication systems;GSM

Abstract. This paper presents an IoT-based vehicle security system that integrates multi-factor authentication for access control and theft prevention. The proposed design employs an NFC reader for license verification as the first authentication layer, followed by fingerprint authentication to verify driver identity. Upon successful verification, a relay module activates the vehicle's motor. In cases of sensor failure or vehicle lending scenarios, a 4×4 matrix keypad provides an alternative authentication path, which is enabled only after explicit owner approval via a mobile application. The system further incorporates a protection mechanism that limits repeated failed fingerprint attempts to reduce unauthorized access. A Flutter-based mobile application, connected to a Python backend server, manages real-time communication between the user and an Arduino controller embedded in a car prototype. Experimental evaluation conducted on the prototype over 100 trials shows a 99.9% NFC read success rate, a 97% fingerprint match success rate for an enrolled user, an average owner approval round-trip time of 2.9 s, a motor activation delay of 700 ms following successful authentication, and $100 \% \log$ delivery completeness on the mobile application. These results demonstrate the feasibility of coordinating multiple authentication mechanisms within a low-cost embedded IoT platform.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 1/5

Python is mentioned only as part of an IoT authentication system, with no evidence about verification, language restriction, safety-oriented toolchains, or DO-178C objectives.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 9 OF 60

Code analysis of saftey-critical and real-time software using ASIS

1999 · Proceedings of the 1999 Annual ACM SIGAda International Conference on Ada

Keywords. tasking, safety-critical, real-time, high integrity, code analysis, Ravenscar profile, Ada language, ASIS

Abstract. The Ravenscar Profile is a restricted tasking profile that supports applications requiring separate threads of control yet would satisfy the certification requirements of high-integrity (safety-critical) real-time systems. If the Ravenscar Profile were to be used for systems having safety-critical and real-time requirements, it would be valuable to demonstrate that the application satisfies the restrictions. Code analysis is an important technique to support this demonstration. Ada Semantic Interface Specification (ASIS) based tools provide an excellent capability for the automatic identification of violations to that set of the Ravenscar Profile restrictions, which can be determined through static code analysis. All but one of these restrictions can be identified using static code analysis using ASIS. This paper provides an approach to building such an ASIS-based tool. This tool might promote the use of automatic tools for the analysis of the Ravenscar Profile and other tasking profiles to support safety-critical and real-time requirements. This paper should be viewed as work in progress.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I4, relevance 4/5

It presents static code analysis for checking restrictions in Ada's Ravenscar Profile, providing a directly relevant safety-critical language-subsetting precedent for RQ1.

Round 3 — rescoring of the included set

include, criterion I4, relevance 4/5

This provides an Ada subsetting precedent: it describes an ASIS static-analysis tool for checking tasking restrictions intended for high-integrity real-time systems, including a limitation that one restriction cannot be identified statically. It can populate the language/tool, restriction rationale, analysis technique, and limitations fields for RQ1/RQ3 comparison.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 10 OF 60

Don’t verify, abstract!

1998 · Proceedings 13th IEEE International Conference on Automated Software Engineering (Cat. No.98EX239)

Keywords. Costs;Humans;Program processors;Sparks;Software safety;Software tools;Writing;Formal specifications;Calculus;Documentation

Abstract. Describes a notation and tool for demonstrating to a third-party certifier that software written in a subset of Ada is safe, and gives some experience of using them on real projects. The thesis underlying the design is that people write adequate code, but that they make design and implementation decisions which can conflict with each other to introduce safety problems. The usual paradigm of formally specifying and then developing and verifying the code is less cost-effective than writing the code and then abstracting it to a level that is suitable for human judgements to be made. This is because there are more people who know how to write good code than those who can write effective formal specifications. The tool processes a formal, or informal, argument that code meets its safety requirements using literate programming and concepts from the refinement calculus developed at Oxford University.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I4, relevance 4/5

It presents a safety-oriented Ada subset and a tool for producing verification arguments, providing a relevant statically typed-language precedent for the review.

Round 3 — rescoring of the included set

include, criterion I4, relevance 4/5

A full reading could populate the language-restriction rationale and verification-approach fields, including how a tool uses literate programming and refinement-calculus concepts to present safety arguments to a certifier. This is useful Ada-subset precedent for RQ1 and RQ3, though the abstract does not establish direct DO-178C relevance.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 11 OF 60

Theory versus practice in real-time computing with the Java/sup TM/ platform

1999 · Proceedings 2nd IEEE International Symposium on Object-Oriented Real-Time Distributed Computing (ISORC'99) (Cat. No.99-61702)

Keywords. Java;Real time systems;Object oriented programming;Computer science;Hardware;Computer languages;Sun;Virtual manufacturing;Software systems;Art

Abstract. The discipline of computer science has always featured a large gap between theory and practice. For example, both the object oriented paradigm and garbage collection techniques go back to at least the 1960s (H. Schorr and W. Waite, 1967; O.J. Dahl and K. Nygard, 1966) but they have only seen widespread commercial acceptance this decade. The Java/sup TM/ programming language has been an important factor leading to the commercial popularization of these techniques. The field of real time computing has suffered from a particularly acute disconnect between theory and practice. Java/sup TM/ technology can be applied to the building of real time systems through a set of standard extensions to the Java/sup TM/ platform. A set of extensions that bring the core benefits of the platform to real time will advance the commercial state of the art. It will help to narrow the gap between theory and practice. The paper explores different aspects of computer systems theory as applied to real time systems. It analyzes whether they are ready to be reduced to practice and standardized.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I3, relevance 3/5

It addresses real-time support in the Java platform, providing relevant background on a managed-language runtime, though it does not specifically evaluate verification or timing determinism.

Round 3 — rescoring of the included set

exclude, criterion E4, relevance 2/5

The abstract presents a broad theory-versus-practice discussion of Java real-time extensions, but gives no specific runtime timing or memory evidence, restrictions, verification approach, or certification relevance to populate the extraction schema.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 12 OF 60

Automated Polyhedron-based TDMA Schedule Design for Predictable Mixed-Criticality MPSoCs

2024 · 2024 27th Euromicro Conference on Digital System Design (DSD)

Keywords. TDMA Scheduling;Real-time;Safety-Critical Scheduling;Freedom of Interference;Mixed-Criticality;Multi-Core System; Schedules;Time division multiple access;Runtime environment;Virtual machine monitors;Processor scheduling;Operating systems;Prediction algorithms;Software;Real-time systems;Security

Abstract. The ongoing trend of centralization of functionality and the resulting integration of previously distributed software components in automotive systems leads to mixed-criticality architectures on those resulting central execution platforms. Multiprocessor system-on-chip architectures provide a powerful platform for centralized execution. A major challenge arises in the safe and secure scheduling of software components on those platforms. Real time tasks can be delayed by interfer-ences occurring by simultaneous access onto shared resources. Meeting deadlines and eliminating contention is often guaranteed by using hypervisors or real time operating systems, which provide a runtime environment to enable tasks to meet their respective deadline in mixed-criticality systems. These runtime environments frequently employ TDMA-based scheduling for a predictable and certifiable execution. In this work, we introduce an algorithm creating scheduling tables for TDMA schedulers that support isolation mechanisms in multicore systems. Using a constructive approach, the resulting scheduling tables support mechanisms for parallel access to shared resources and windows for exclusive execution. Our approach is validated by reconstructing previously generated and valid synthetic scheduling tables. We achieve a high success rate in all cases for a processor utilization of up to 80 % with an algorithm runtime of 10 seconds.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E3, relevance 2/5

The work concerns multicore TDMA scheduling and shared-resource isolation, not Python or a comparable language, language-level runtime timing, or a verification approach relevant to the review questions.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 13 OF 60

Digital Design Interactive Real-Time Color Theory Application in Digital Education

2025 · 2025 IEEE Digital Education and MOOCS Conference (DEMOcon)

Keywords. interactive color analysis;real-time image processing;web-based design tools;digital education;color theory application;flask python web app;multimedia learning technologies; Hands;Visualization;Image color analysis;Webcams;Education;Prototypes;Real-time systems;Design tools;Usability;Videos

Abstract. This paper presents a practical implementation of color theory using a web application built with Python and the Flask framework and its implication on digital education in multimedia technologies. The system enables automatic extraction of dominant color palettes from both static images and live webcam streams. Leveraging libraries such as OpenCV, Pillow, and Color Thief, the application supports real-time analysis and visualization of RGB, HEX and CMYK color formats. Users can upload images or use their device's camera to generate personalized palettes, providing immediate feedback through a dynamic web interface. The implementation also demonstrates asynchronous communication between client and server using AJAX calls to periodically refresh color data without page reload. This architecture ensures an interactive and fluid experience, simulating real-world conditions relevant to digital design. Moreover, this approach showcases the synergy between computational tools and design principles, validating the feasibility of integrating real-time image analysis into creative workflows. The prototype developed in this work is fully functional and integrates live video analysis, automatic palette extraction, and format conversion into a unified web application. Unlike existing tools that are limited to static image processing or API-only services, this solution provides a more efficient method of color analysis. By enabling dynamic interaction through both image upload and live webcam input, it establishes a practical foundation for a new category of interactive design tools, more accessible compared to existing implementations. This allows for a better understanding of multimedia principles by undergraduate students.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 1/5

The paper describes an educational color-analysis application, not Python runtime timing, verification, language restriction, or safety-toolchain feasibility.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 14 OF 60

Safety-Centric Design of Distributed Embedded Avionics

2008 · 2008 IEEE National Aerospace and Electronics Conference

Keywords. Aerospace electronics;Computer architecture;Control systems;Actuators;Safety;Runtime;Costs;Embedded computing;Unmanned aerial vehicles;Sensor systems

Abstract. This paper describes a methodology for safety-centric development of distributed embedded avionics realized as sense-actuate-control (SAC) networks. The methodology has consists of two parts. First, formal methods are used in defining and deriving families of SAC node architectures. This methodology eliminates redundant verification and validation (V&V) efforts across members of the same family of architectures. Second, proof-directed run-time error-monitor generation methodology is presented. This methodology links design-time verification with run-time error checking. Robust error monitors can be derived and reused across the members of a family of architectures.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I5, relevance 3/5

The avionics study describes a substantive formal-verification and runtime-monitoring methodology that may inform how verification evidence is produced for airborne software, though it does not address Python or identify specific DO-178C objectives.

Round 2 — re-read of borderline records

exclude, criterion E4, relevance 2/5

The paper concerns avionics architecture derivation and reusable runtime error monitors, not language restrictions, program-analysis tools, managed-runtime behavior, or DO-178C objective evidence. It would not credibly populate an extraction field bearing on the research questions.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 15 OF 60

Semi-formal approach for validating compiler for safety critical software in airborne systems

2015 · 2015 2nd International Conference on Computing for Sustainable Global Development (INDIACom)

Keywords. Compiler validation;Knowledge database;Program analysis;Object code analysis;Semi-formal Representation;Source code analysis; Assembly;Databases;Program processors;Standards;Manuals;Safety

Abstract. For safety critical systems, porting the correct object code to perform the safety function is very critical. So apart from verifying the embedded software at hardware, software and system levels, safety critical systems need to be tested and validated even at the object code level [1,2,3] (ref: DO 178, Section 6.4.4.2). This paper proposes a novel semi-formal approach for validating compiler used for safety critical software for airborne systems. The novel approach is demonstrated as a case study for Microtec cross-compiler used for generating the object code for Motorola 68060 for indigenously developed Stall warning and aircraft interface computer system software. Program Analysis technique is used to validate the cross-compiler. The technique is incorporated in an in-house tool which integrates the manual knowledge database to carry out the compiler validation. This tool provides partial automation of the verification process which will be used to provide proof as per the RTCA DO-178B guidelines [3].

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I5, relevance 3/5

Although it does not address Python or a comparable language, it describes producing compiler-validation evidence for an airborne-software standard, making it relevant background for RQ3.

Round 2 — re-read of borderline records

include, criterion I5, relevance 4/5

The study describes a compiler-validation approach and its application to airborne software, including how object-code analysis and an in-house tool are used to produce DO-178B evidence. This can populate the verification approach, tool, and DO-178 relevance extraction fields.

Round 3 — rescoring of the included set

include, criterion I5, relevance 4/5

A full reading could populate the verification approach, tool and trusted-evidence process, and stated DO-178 objectives: it describes semi-formal compiler validation using program analysis and an in-house tool, demonstrated on an airborne-system case study.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 16 OF 60

GPU-Accelerated Multi-Strategy Simulation Framework for Real-Time High-Frequency Trading

2025 · 2025 IEEE Pune Section International Conference (PuneCon)

Keywords. GPU acceleration;high-frequency trading;backtesting;Numba;CuPy;real-time analytics;order book simulation;microstructure modeling;financial computing;multistrategy optimization;performance visualization; High performance computing;Heuristic algorithms;Computational modeling;Scalability;Graphics processing units;Finance;Transforms;Parallel processing;Real-time systems;Optimization

Abstract. High-Frequency Trading (HFT) demands microsecond-level decision-making and realistic backtesting at scale. Traditional CPU-based simulators are limited by sequential execution and lack microstructural accuracy. This paper presents a GPU-Accelerated Dynamic Multi-Strategy Trading Simulation Framework that enables real-time, parallel evaluation of multiple algorithms under authentic market conditions. Utilizing Numba and CuPy for GPU parallelism, the system achieves significant speedups while processing largescale tick data from NASDAQ and cryptocurrency markets. The simulator models key microstructural effects-latency, slippage, order book depth, and partial fills-to enhance execution realism. Real-time visualization through Plotly and Matplotlib delivers interactive analytics on PnL, Sharpe ratios, and latency distributions. Experimental results show over 7 x performance improvement compared to CPU-based systems and reveal novel interactions among concurrent trading strategies. The framework bridges high-performance computing (HPC) and quantitative finance, establishing a scalable foundation for research in GPU-driven algorithmic trading.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 1/5

The paper evaluates GPU-accelerated trading simulation performance, not language-runtime timing determinism, verification, language restriction, or safety-toolchain feasibility.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 17 OF 60

Chip-multiprocessor hardware locks for safety-critical Java

2013 · Proceedings of the 11th International Workshop on Java Technologies for Real-Time and Embedded Systems

Keywords. synchronization, safety-critical Java, hardware locks

Abstract. Accessing shared resources in multicore systems is usually protected by a software locking mechanism, which itself is implemented through atomic operations. This can result in a large synchronization overhead, which, in the context of real-time systems, increases the worst-case execution time and may void a task set's schedulability. In this paper we present a hardware locking mechanism to reduce the synchronization overhead. The solution is implemented for the chip-multiprocessor version of the Java Optimized Processor in the context of safety-critical Java. The implementation is compared to a software solution. The performance and the hardware cost are evaluated.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 1/5

The paper evaluates hardware-based synchronization and its performance cost, not language-runtime timing, verification, language restriction, or DO-178C objectives relevant to the review.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 18 OF 60

Real-Time Face Detection–Based Entry Counting System for Attendance Monitoring

2026 · 2026 9th International Conference on Electronics, Materials Engineering & Nano-Technology (IEMENTech)

Keywords. Attendance monitoring;Computer vision;Face detection;Haar Cascade Classifier;Real-time systems; Accuracy;Fluctuations;Event detection;Face recognition;Manuals;Real-time systems;Face detection;Logic;Transient analysis;Monitoring

Abstract. This paper presents a real-time face detection-based entry counting system for automated attendance monitoring. The system is implemented using Python and OpenCV and employs Haar Cascade Classifiers for real-time facial detection. A temporal state-based decision logic ensures that each person is counted only once during an entry event, thereby reducing duplicate counts caused by motion fluctuations and transient detection loss. Experimental evaluation conducted across classroom, corridor and office environments demonstrates an average detection accuracy of 96.7 % with a mean processing rate of 21.4 FPS. Comparative analysis with HOG+SVM, MobileNet-SSD and MTCNN detectors shows that the proposed method provides an efficient trade-off between accuracy, computational cost and deployment simplicity, making it suitable for lightweight real-time attendance monitoring applications.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 1/5

The paper uses Python for a face-detection attendance application and reports application throughput, but does not address runtime timing determinism, verification, language restriction, or safety-oriented toolchain feasibility.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 19 OF 60

A Time Predictable Java Processor

2006 · Proceedings of the Design Automation & Test in Europe Conference

Keywords. Java;Pipelines;Computer architecture;Real time systems;Process design;Virtual machining;Decoding;Design engineering;Hardware;Performance analysis

Abstract. This paper presents a Java processor, called JOP, designed for time-predictable execution of real-time tasks. JOP is the implementation of the Java virtual machine in hardware. We propose a processor architecture that favors low worst-case execution time (WCET) over average case performance. The resulting processor is an easy target for the low-level WCET analysis

---

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 1/5

The paper concerns a hardware processor architecture for WCET predictability, not language-runtime timing behaviour, verification, language restriction, or DO-178C objectives relevant to the review.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 20 OF 60

Towards Balancing Determinism, Memory Consumption and Throughput for RTSJ-Based Real-Time Applications

2011 · 2011 11th International Conference on Quality Software

Keywords. real-time applications;throughput;determinism;memory consumption;stochastic process;integer programming; Instruction sets;Real time systems;Throughput;Time factors;Memory management;Java;Stochastic processes

Abstract. Determinism, memory consumption and throughput are three important performance indicators for RTSJ-based real-time applications, but they often interact and conflict with each other. Manually balancing these performance indicators is often time-consuming. Therefore, there is the need to clarify the relationship among the performance metrics and make trade-off automatically. In this paper, we abstract real-time thread properties relating to these performance indicators and analyze their relationship. Based on the analysis, an automatic configuration framework to balance the performance metrics is proposed. In this framework, a stochastic process is developed to represent real-time threads' determinism. And real-time application's throughput and memory consumption is quantified with these thread properties as parameters. Experiment results based on Sweet Factory application show that our approach could optimize memory consumption and throughput while guaranteeing determinism effectively.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I3, relevance 3/5

The RTSJ thread-determinism analysis provides relevant timing and execution-model evidence for a managed language runtime, although it does not address verification or certification objectives directly.

Round 3 — rescoring of the included set

include, criterion I3, relevance 4/5

This could populate the approach, property classes, and evaluation fields by analysing real-time thread properties and trading off determinism, memory consumption, and throughput. It provides managed-Java real-time precedent relevant to RQ3, though it does not directly address certification objectives.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 21 OF 60

Rationale for Ada 2005: 4 tasking and real-time

2005 · Ada User Journal

No abstract available — this record was judged on its title alone.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I3, relevance 3/5

The title indicates language-level tasking and real-time material that could provide background on execution-model and timing considerations relevant to RQ3, though it does not indicate direct treatment of verification or DO-178C objectives.

Round 2 — re-read of borderline records

include, criterion I3, relevance 4/5

The title indicates a discussion of Ada's tasking and real-time language features, which could inform the extraction fields on language constructs and restrictions and provide a comparator precedent for RQ1/RQ3.

Round 3 — rescoring of the included set

exclude, criterion E6, relevance 3/5

The numbered topic appears to be a chapter in the book *Rationale for Ada 2005*, which is excluded under E6. Its Ada tasking and real-time subject could offer comparator context, but the title alone does not indicate a verification or language-restriction approach.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 22 OF 60

Dynamic Region Ownership for Concurrency Safety

2025 · Proc. ACM Program. Lang.

Keywords. Ownership, Concurrency Safety, Dynamic languages, Python

Abstract. The ways in which the components of a program interact with each other in a concurrent setting can be considerably more complex than in a sequential setting. The core problem is unrestricted shared mutable state. An alternative to unrestricted shared mutable state is to restrict the sharing using Ownership. Ownership can turn what would have been a race into a deterministic failure that can be explained to the programmer. However, Ownership has predominantly taken place in statically typed languages.In this paper, we explore retrofitting an existing dynamically typed programming language with an ownership model based on regions. Our core aim is to provide safe concurrency, that is, the ownership model should provide deterministic dynamic failures of ownership that can be explained to the programmer. We present a dynamic model of ownership that provides ownership of groups objects called regions. We provide dynamic enforcement of our region discipline, which we have implemented in a simple interpreter that provides a Pythonlike syntax and semantics, and report on our first steps into integrating it into an existing language, Python.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I3, relevance 5/5

The paper develops and dynamically enforces an ownership model for a dynamically typed language, including Python, making it directly relevant to concurrency-safety restrictions and verification of Python.

Round 3 — rescoring of the included set

include, criterion I3, relevance 5/5

This would populate the language/tool, enforcement approach, concurrency property, and language restriction fields: it explores region-based ownership in a dynamically typed language, including Python, to turn unsafe sharing into deterministic failures. It directly informs RQ1's treatment of concurrency races and dynamic-language restrictions.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 23 OF 60

Framework for the Design of Java Firm Real-time Systems Oriented to the Generation of Timing Behaviour Models

2013 · 2013 39th Euromicro Conference on Software Engineering and Advanced Applications

Keywords. Real-time;RTSJ;schedulability analysis; Real-time systems;Synchronization;Java;Message systems;Object oriented modeling;Metrology;Analytical models

Abstract. This work proposes a framework for the design of complex Java firm real-time applications that run on embedded systems. The framework simplifies the real-time patterns included in the Real-Time Specification for Java (RTSJ) with the aim of systematizing the process of modelling and configuring the scheduling of real-time object-oriented applications. Moreover, with the proposed strategy, the designer of a real-time application can automatically obtain the timing behaviour model of the application, which is used to analyse and/or configure its schedulability so that it can be executed on an embedded system meeting its firm real-time requirements. The framework supports the design of applications whose real-time tasks interact asynchronously with non real-time ones.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I3, relevance 3/5

The Java RTSJ framework models timing and schedulability in a managed-language execution context, providing relevant background for RQ3, though it does not directly address DO-178C objectives or verification.

Round 3 — rescoring of the included set

include, criterion I3, relevance 4/5

The framework provides a Java/RTSJ approach to modeling application timing and analysing schedulability. A full reading could populate the analysis technique, language-level real-time mechanisms, and timing-property fields as precedent for RQ3.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 24 OF 60

SGXPy: Protecting Integrity of Python Applications with Intel SGX

2019 · 2019 26th Asia-Pacific Software Engineering Conference (APSEC)

Keywords. SGX, Python, trusted computing, integrity; Cloud computing;Trusted computing;Runtime environment;File systems;Process control;Libraries;Web servers;Software;Python;Testing

Abstract. Python is the programming language of choice for many data scientists, and thus widely used in cloud computing platforms. Untrusted cloud environments have imposed challenges to the security of Python applications. Intel SGX (Intel Software Guard eXtensions) provides an encrypted enclave for securing applications, and a library OS technology can be adopted to run legacy applications inside these enclaves. However, this technology has some limitations: (i) It is difficult to ensure the integrity of Python applications as a result of the complex dependencies among modules. (ii) Python applications often spawn new processes, and file access permissions need to be handled separately in the parent-child process. To address these limitations, we present SGXPy (SGX Python), an integrity preserving tool for Python applications. The design of SGXPy makes it possible to obtain dependencies of applications and assign file access permissions among processes automatically: (i) During the build stage, SGXPy constructs dependency manifests of Python applications based on the ptrace mechanism. (ii) To enhance access control among processes, SGXPy utilizes process introspection to cascading manifests for each process. With the proposed framework, sophisticated Python applications such as NumPy and a web server can now run unmodified with the library OS. We present a series of experiments to evaluate performance overheads of Python applications in SGX. Our evaluation of NumPy submodules shows SGXPy can pass 97.60% of unit testing, even with the isolated environment and limited memory of SGX.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 1/5

The work addresses application integrity and access control in Intel SGX rather than safety-oriented verification, language restriction, timing determinism, or DO-178C objectives.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 25 OF 60

Performance and productivity of parallel python programming: a study with a CFD test case

2015 · Proceedings of the 5th Workshop on Python for High-Performance and Scientific Computing

Abstract. The programming language Python is widely used to create rapidly compact software. However, compared to low-level programming languages like C or Fortran low performance is preventing its use for HPC applications. Efficient parallel programming of multi-core systems and graphic cards is generally a complex task. Python with add-ons might provide a simple approach to program those systems. This paper evaluates the performance of Python implementations with different libraries and compares it to implementations in C or Fortran. As a test case from the field of computational fluid dynamics (CFD) a part of a rotor simulation code was selected. Fortran versions of this code were available for use on single-core, multi-core and graphic-card systems. For all these computer systems, multiple compact versions of the code were implemented in Python with different libraries. For performance analysis of the rotor simulation kernel, a performance model was developed. This model was then employed to assess the performance reached with the different implementations. Performance tests showed that an implementation with Python syntax is six times slower than Fortran on single-core systems. The performance on multi-core systems and graphic cards is about a tenth of the Fortran implementations. A higher performance was achieved by a hybrid implementation in C and Python using Cython. The latter reached about half of the performance of the Fortran implementation.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 1/5

The record concerns general-purpose performance benchmarking for CFD, not runtime timing determinism, verification, language restriction, safety, or toolchain feasibility relevant to the review questions.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 26 OF 60

Autograding Python Code with the Pedal Framework: Feedback Beyond Unit Tests

2024 · Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 2

Keywords. autograder, autograding, blockpy, cs1, dynamic analysis, feedback, gpt, gradescope, pedal, python, static analysis, typechecking, unit tests, webcat, wheats and chaffs

Abstract. The ever-increasing enrollments in programming courses has driven the need for sophisticated grading tools that can provide students with precise, insightful, and timely feedback. This SIGCSE workshop presents an interactive session on our powerful, open-source Python autograding framework, Pedal. As a free library, Pedal is available on a wide range of grading platforms, including GradeScope and BlockPy - anything that allows installation of a pure Python library. Pedal supports but goes beyond traditional unit testing, providing advanced code analysis techniques, such as type checking, liveness checking, structural code pattern matching, and more. Pedal has a large collection of assertions to evaluate dynamic program traces, query the Abstract Syntax Tree, re-execute student code under varying conditions, mock inputs, and capture outputs. Every feedback message is treated as a first-class object, empowering educators to fine-tune feedback as desired. Pedal is not just a toolset but a comprehensive pipeline optimized for feedback selection, resolution, and evaluation. With functionalities like command-line batch execution, exhaustive metadata tracking, and A/B testing, educators and researchers can analyze and refine their feedback strategies. With Pedal's successful deployment across multiple courses and institutions over the years, this workshop will offer attendees firsthand experience and a plethora of real-world examples. By the end of this workshop, participants will be proficient in leveraging Pedal, even venturing into the realm of creating interactive activities using the framework.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 2/5

Although it presents Python code-analysis techniques, the work focuses on educational autograding and feedback, not safety, formal verification, language restriction, or safety-oriented toolchain feasibility.

Round 2 — re-read of borderline records

include, criterion I3, relevance 5/5

A full reading could populate the approach and property-classes fields for Python analysis, including static checks and dynamic trace analysis. Its educational purpose does not negate the technical relevance of these techniques.

Round 3 — rescoring of the included set

include, criterion I3, relevance 5/5

A full reading could populate the Python tool and analysis-approach fields, including static checks and dynamic-trace analysis, providing a comparator for RQ2. Its focus is educational feedback rather than assurance, so it is unlikely to establish soundness or certification claims.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 27 OF 60

An Offline-First Raspberry Pi-based IoT Health Monitoring System for Low-Connectivity Environments with a Real-Time Dashboard

2026 · 2026 5th International Conference on Computer Networks, Big Data and IoT (ICCBI)

Keywords. Internet of Things;Raspberry Pi;health monitoring;offline-first system;SpO2;heart rate;edge computing;real-time dashboard; Internet of Things;Monitoring;Occupational health;Temperature;Heart rate;Triples (Data structure);Medical services;Visualization;Printing;Prototypes

Abstract. Internet of Things (IoT)-based healthcare monitoring systems commonly depend on cloud connectivity for data storage, visualization, and remote access. This dependence limits their reliability in rural clinics, emergency camps, disaster-affected regions, and other low-connectivity environments. This paper presents the design, implementation, and preliminary evaluation of an offline-first IoT health monitoring prototype built on a Raspberry Pi platform. The system acquires four physiological and environmental parameters: heart rate, peripheral oxygen saturation (SpO2), body temperature, and ambient temperature. It integrates an MLX90614 infrared temperature sensor, a MAX30102 pulse oximeter module, a Python-based acquisition layer, a Node.js/Express local backend, local JSON storage, and a React dashboard served through a Raspberry Pi-hosted Wi-Fi hotspot. Users can access the dashboard from Wi-Fi-enabled smartphones, tablets, or laptops by entering the local IP address in a browser without requiring internet access. Quantitative validation against commercial reference devices produced mean absolute errors of 2 BPM for heart rate, 0.6% for SpO2, and 0.2C for body temperature under preliminary resting measurements. Benchmarking against recent IoT and edge-health monitoring studies shows that the proposed system contributes a compact, low-cost, multi-parameter, offline-first alternative for resource-constrained healthcare deployment. The results indicate that local edge processing and dashboard visualization can improve continuity, privacy, and usability where cloud-dependent health monitoring is impractical.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 1/5

The paper uses Python in an IoT health-monitoring prototype but does not address verification, language restriction, runtime timing behaviour, safety toolchains, or DO-178C objectives.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 28 OF 60

From Formal Specification to Verified Implementation

2024 · 2024 AIAA DATC/IEEE 43rd Digital Avionics Systems Conference (DASC)

Keywords. formal specification;formal verification;high-assurance software; Fans;Autonomous systems;Software algorithms;Software;Software reliability;Formal specifications;Sparks;Security;Testing;Software development management

Abstract. With increasingly complex algorithms used in autonomous systems, ensuring the safety, security, and correctness of software is paramount. We present a process for developing high-quality software, encompassing formal specification, implementation, and verification processes. Our approach focuses on developing high-assurance systems using PVS for formal specification, SPARK Ada for implementation, and the SPARK Pro toolset for formal verification, augmented by FAN (Friendly Argument Notation) arguments. This methodology is demonstrated through a detailed case study involving the design and verification of a polygon merge algorithm for OpenUxAS. In our case study, we define and prove critical properties of the algorithm, such as the preservation of all input points in the output and the containment of output points within the original polygons or generated holes. The formal specification has been added to the NASA PVS library, expanding the formalized knowledge about polygons. This integrated approach offers a higher level of assurance than conventional methods, addressing the limitations of traditional testing and debugging practices. By combining PVS, SPARK Ada, and FAN, we streamline the verification process while maintaining correctness assurance, contributing to the advancement of software development practices for safety-critical and autonomous systems.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I4, relevance 4/5

It presents formal verification of safety-critical software using SPARK Ada, providing a relevant statically typed-language precedent for the review.

Round 3 — rescoring of the included set

include, criterion I4, relevance 4/5

This provides a SPARK comparator for RQ1/RQ3 and can populate the verification approach, formally proven properties, developer obligations, and case-study evaluation fields. It describes formal verification of a safety-critical algorithm, though it is not direct evidence about Python or DO-178C objectives.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 29 OF 60

Formal Verification for Security Certification: From a First Success to Sustainable Industrial Usage

2026 · Lecture Notes in Computer Science

Abstract. Abstract The application of formal methods in an industrial context remains a delicate and time-consuming task, and each successful project contributes to improving productivity. This experience report summarizes five years of applying formal verification for Common Criteria-based security certification of a JavaCard Virtual Machine at Thales. Since the first successful formal verification—directly over the source code with the verification platform—was achieved in 2021, five highest-level Common Criteria certifications (at EAL6 and EAL7 levels) have been obtained during the past five years. The strong guarantees of functional correctness and security provided by source code-based verification are essential for security-critical products such as smart cards. This paper traces the evolution of the project from an initial success to the continuous application of formal verification, and discusses the main challenges, applied solutions, and opportunities for further improvement.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 2/5

The paper reports formal verification for security certification, but its JavaCard and Common Criteria focus does not address Python or a comparable dynamic language, nor DO-178C objectives or evidence.

Round 2 — re-read of borderline records

include, criterion I4, relevance 3/5

A full reading could provide comparator evidence on the verification approach, properties established, trusted verification platform, and industrial challenges for a managed-language virtual machine. Its Common Criteria context is indirect evidence for the DO-178C questions, so it is corroborative rather than a primary source.

Round 3 — rescoring of the included set

include, criterion I4, relevance 3/5

A full reading could populate fields on the verification approach, properties established, and industrial experience applying source-code verification to a managed virtual machine. Its Common Criteria context makes it corroborative rather than direct evidence about DO-178C objectives or Java runtime certification precedent.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 30 OF 60

Symbolic and Analytic Techniques for Resource Analysis of Java Bytecode

2010 · Trustworthly Global Computing

No abstract available — this record was judged on its title alone.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 2/5

The title indicates resource-analysis techniques for Java bytecode, but gives no evidence of a safety, verification-toolchain, DO-178C, or dynamic-language connection relevant to the review questions.

Round 2 — re-read of borderline records

include, criterion I4, relevance 4/5

A full reading could describe analysis techniques and the resource properties or bounds they establish for Java bytecode, providing managed-language comparator evidence relevant to RQ3. The title alone does not establish a direct DO-178C connection.

Round 3 — rescoring of the included set

include, criterion I3, relevance 4/5

This could provide a verification approach and evidence about resource properties for Java bytecode, a managed-language comparator relevant to RQ3. A full text may also specify the analysis technique, properties established, assumptions, and limitations.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 31 OF 60

Reusable Libraries for Safety-Critical Java

2014 · 2014 IEEE 17th International Symposium on Object/Component/Service-Oriented Real-Time Distributed Computing

Keywords. Real-time systems;Java;Safety-critical systems;Safety-critical Java; Libraries;Java;Memory management;Arrays time systems;Resource management;Instruction sets

Abstract. The large collection of Java class libraries is a main factor of the success of Java. However, these libraries assume that a garbage-collected heap is used. Safety-critical Java uses scope-based memory areas instead of a garbage-collected heap. Therefore, the Java class libraries are problematic to use in safety-critical Java. We have identified common programming patterns in the Java class libraries that make them unsuitable for safety-critical Java. We propose ways to improve the libraries to avoid the impact of the identified problematic patterns. We illustrate these changes by implementing a total of five scope-safe classes from commonly used libraries.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E3, relevance 2/5

The paper concerns adapting Java libraries to safety-critical memory areas, but does not address Python or a comparable dynamic language, verification evidence, or DO-178C objectives.

Round 2 — re-read of borderline records

include, criterion I4, relevance 4/5

The study could populate extraction fields on language/runtime restrictions and their rationale: it identifies library patterns incompatible with safety-critical Java's memory model and proposes scope-safe alternatives. This provides managed-language precedent relevant to RQ3.

Round 3 — rescoring of the included set

include, criterion I3, relevance 4/5

This would help populate the restriction and rationale fields by identifying library patterns incompatible with scope-based memory and describing changes to make classes scope-safe. It also provides managed-language precedent relevant to RQ3 on memory-management constraints in safety-critical Java.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 32 OF 60

The Epistemic Engine: A Deterministic Structural Auditor for Institutional Discourse

2026 · 2026 IEEE International Conference on Engineering, Technology & Management (ICETM)

Keywords. epistemic auditing;institutional discourse;deterministic classification;authority laundering;kernel violation;register-mismatch gradient;falsifiability;NLP benchmarks;bias detection;SEC disclosure analysis;MAFALDA;SemE Kappa;inter-model agreement; Modeling;Engines;Kernel;Generative Pre-trained transformer;Printing;Registers;Cleaning;Ca detection;Large language models

Abstract. Institutional discourse is increasingly produced and read with AI assistance, yet no tool currently exists to evaluate its structural epistemic properties automatically and reproducibly. This paper presents the Epistemic Engine, a deterministic, layered auditing system that classifies how institutional text makes its claims. The system implements a six-layer architecture: structural signal extraction, language game classification, bias gate detection, authority-laundering identification, learning posture derivation, and two-axis rhetorical coordinate placement. Implemented as a model-agnostic Python pipeline deployed across two AI platforms (OpenAI GPT o3 and Google Gemini), the engine uses lexicon-based density computation with threshold-based deterministic classification. Validation against a 69-text corpus comprising 67 documents across six sub-corpora and 2 calibration seeds produced three principal findings. First, a systematic pattern of kernel violations: both LLM platforms correctly compute linguistic signal densities at the input layer but fail to execute the deterministic co-occurrence logic required to flag authority laundering and bureaucratic masking, a disconnect documented across three independent corpora. Second, a register-mismatch gradient: the engine achieves 6/6 gate classification on synthetic institutional texts and consistent structural bifurcation on SEC regulatory filings, but produces $\kappa=0.053$ (Gemini) and $\kappa=0.000$ (GPT) against the MAFALDA informal-argumentation corpus, empirically establishing that institutional structural constructs (evasion, authority laundering, defensive routines) do not map onto general-purpose argumentation fallacies. Third, a scale contrast with institutional annotation precedent: the European Commission's JRC campaigns required over seventy scholars annotating 2,500 documents across nine languages with months of calibration and persistent expert disagreement; the deterministic kernel produces reproduci

---

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 1/5

Although implemented in Python, the work concerns auditing institutional discourse and does not address language runtime behaviour, verification, language restriction, safety, or DO-178C objectives.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 33 OF 60

Deterministic Java in tiny embedded systems

2001 · Fourth IEEE International Symposium on Object-Oriented Real-Time Distributed Computing. ISORC 2001

Keywords. Java;Embedded system;Computer languages;Memory management;Time to market;Programming profession;Assembly;Kernel;Automatic testing;Performance evaluation

Abstract. As embedded systems become more and more complex, and the time to market becomes shorter, there is a need in the embedded systems community to find better programming languages that let the programmers develop correct code faster. The programming languages used today, typically C and/or Assemblers, are just too error-prone. The Java technology has therefore gained a lot of interest from developers of embedded systems in the last few years. We propose an approach based on compiling Java into native machine code via C as an intermediate language. The C code generation process should also add close interaction with a fully pre-emptive incremental garbage collector and a small and efficient real time kernel. Tests performed on a small 8-bit microprocessor show that it is possible to use a modern object oriented language with automatic memory management, such as Java, and yet generate fully predictable code that can be run in very small devices with severe memory constraints.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I3, relevance 4/5

The paper evaluates Java runtime timing predictability with automatic memory management, providing relevant managed-language precedent for RQ3.

Round 3 — rescoring of the included set

include, criterion I4, relevance 4/5

A full reading could populate the translation-to-intermediate-form, runtime memory-management, and predictability fields, including the role of a garbage collector and real-time kernel. This provides managed-language precedent relevant to RQ3 on whether deterministic behavior is attainable under severe embedded constraints.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 34 OF 60

Practical experiences of applying source-level WCET flow analysis on industrial code

2010 · Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)

No abstract available — this record was judged on its title alone.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E3, relevance 2/5

The title identifies source-level WCET analysis but gives no evidence that the work concerns Python or another comparable dynamic language, language-runtime timing, or DO-178C objectives and verification evidence.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 35 OF 60

Preemptive type checking in dynamically typed languages

2011 · Lecture Notes in Computer Science

Keywords. type checking / Python-like dynamic languages

No abstract available — this record was judged on its title alone.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I3, relevance 5/5

The record presents type checking for dynamically typed languages, directly relevant to the review's static/type-checking question and applicable to Python-like languages.

Round 3 — rescoring of the included set

include, criterion I3, relevance 5/5

The title identifies a type-checking technique for dynamically typed languages, which could populate the approach, property classes, and any required type restrictions or annotations fields. Its relevance to safety-critical verification is not stated, but the technique directly bears on RQ2.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 36 OF 60

Towards a type-based abstract semantics for Python

2025

Keywords. Python static analysis / type inference

No abstract available — this record was judged on its title alone.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I3, relevance 5/5

The record directly concerns a formal static-analysis and type-inference approach for Python, which can inform the review's comparison of Python verification and analysis tools.

Round 3 — rescoring of the included set

include, criterion I3, relevance 5/5

This appears to develop a static-analysis or type-inference framework for Python. A full reading could populate the analysis technique, properties addressed, and soundness or completeness fields relevant to RQ2.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 37 OF 60

An approach to the certification of autonomous systems

2014 · 9th IET International Conference on System Safety and Cyber Security (2014)

Keywords. Certification;Formal Methods;Autonomy;Safety;Security

Abstract. This paper outlines the approach taken to produce a certified autonomous vehicle under a project funded by the UK Technology Strategy Board. The aim of the project was to show that it is possible to automate system design validation as well as the verification of an implementation in software. By doing so, we showed considerable potential for reduced time and cost while still being able to meet certification requirements. The project used a Maritime Autonomous System case study and used the COLREGS [1], but applied the stringent DO-178C [2] software safety assurance requirements from the aerospace domain. The project focussed on the use of Formal Methods and hence exploited the technology specific supplement DO-333 [3]. The project highlights the applicability of the approach to safety critical autonomous systems in any domain.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I5, relevance 4/5

The paper describes producing software verification evidence using formal methods under DO-178C/DO-333, directly informing how certification objectives may be supported.

Round 3 — rescoring of the included set

include, criterion I5, relevance 4/5

The paper describes a certification case and use of formal methods to verify an implementation, which could populate the verification approach and DO-178C/DO-333 relevance fields. It offers evidence about certification practice, though its abstract does not establish that the approach applies specifically to Python or another managed-language runtime.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 38 OF 60

Real time SAR Ship Detection using novel SarNeDe method

2021 · 2021 IEEE International Geoscience and Remote Sensing Symposium IGARSS

Keywords. ship detection;SAR;deep learning;convolution neural network (CNN); Deep learning;Neural networks;Graphics processing units;Speckle;Predictive models;Radar polarimetry;Real-time systems

Abstract. Many research works using deep learning techniques for automatic ship detection from SAR images have good detection accuracy. But the main problem in these methods is false detection, mostly due to speckle presence. Therefore, we propose a new deep learning model with a novel SarNeDe preprocessing stage to address this problem. We are introducing a deep learning architecture to detect and localize ships in the SAR image. First, generate a three-channel SarNeDe image in the preprocessing step. Then, this SarNeDe image is used to train the model to predict the ship's position in the SAR image. We experimented on the public SAR ship detection dataset (SSDD) and Dataset of Ship Detection for Deep Learning under Complex Backgrounds (SDCD) to validate the proposed method's feasibility. We used python 3.5 for coding with the Keras framework in the NVIDIA Tesla K80 GPU hardware platform. The experimental results indicated that our proposed method's ship detection accuracy has increased with reduced false detection percentage.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 1/5

The work uses Python for SAR ship detection, but its real-time context concerns the detection application rather than language-runtime timing, and it provides no safety, verification, restriction, or toolchain evidence.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 39 OF 60

Model-based avionic systems testing for the airbus family

2018 · 2018 IEEE 23rd European Test Symposium (ETS)

Keywords. model-based testing;test automation;model-based systems engineering;avionic systems; Testing;Switches;Aerospace electronics;Standards;Atmospheric modeling;Biological system modeling

Abstract. This paper is about practical verification of Airbus avionic systems during type certification, with special focus on automated testing. The material is based on test and verification services performed for Airbus by a spinoff company of the University of Bremen, as well as on consultancy services delivered by our research group to Airbus and its suppliers. In the context of model-based systems engineering, the test automation approach is currently shifting from manual test procedure programming to model-based testing (MBT), where test cases are automatically identified in models describing the application behavior, allowing for automated test data calculation and test procedure generation. We describe the situations where today's MBT technology is already adequate to increase the effectiveness of automated testing in industry. In addition, we describe some open challenges arising from practical avionic systems testing, where satisfactory solutions still require some research effort.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I5, relevance 3/5

It describes how automated testing produces verification evidence for avionics type certification, making it relevant background for RQ3, though it does not address Python or specific DO-178C objectives.

Round 2 — re-read of borderline records

include, criterion I5, relevance 4/5

The paper describes a concrete verification approach—model-based testing that generates test cases, test data, and procedures—and reports where it is adequate and where challenges remain. This can inform RQ3 about verification activities and their practical limits in airborne certification, though it does not address Python or name specific DO-178C objectives.

Round 3 — rescoring of the included set

include, criterion I5, relevance 3/5

The paper could populate the verification-approach and certification-relevance fields: it describes model-based testing that derives test cases, data and procedures, as well as practical challenges. Its contribution is indirect because the abstract does not identify specific DO-178C objectives or address Python or runtime constraints.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 40 OF 60

Real-Time Operating Systems' Compliance With MISRA-C Coding Standard: A Comprehensive Study

2024 · IEEE Access

Abstract. Ensuring the security and safety of a real-time operating system (RTOS) is crucial for protecting against potential cyber-attacks. A robust system can provide reliable and uninterrupted operations in the system as well as protect sensitive data and critical functions. To help programmers develop robust programs and systems, multiple secure coding standards have been developed by different organizations. These coding standards offer guidelines for writing secure code to eliminate vulnerabilities and prevent undefined behaviors from happening. The goal of this research is to understand whether several free common RTOSs follow these secure coding standards. We chose MISRA-C:2012 for our experiment because it is a widely recognized coding standard specifically designed for the safe use of the C programming language in critical systems. To investigate whether existing RTOSs adhere to MISRA-C:2012, we utilized a static code analysis tool, Cppcheck, to analyze 16 open-source RTOSs (written in C) for compliance with 153 rules in MISRA-C:2012. The results indicate that most of the RTOSs conform to mandatory rules of MISRA-C. However, quite a few of the required rules are still violated by many of the RTOSs. Some of the violated rules can potentially result in critical issues in the RTOSs, but most of them can be avoided by simply writing better and safer C code. This paper discusses the major issues in the implementation of RTOSs found in our experiment. The analysis of the results will help programmers and researchers understand the primary concerns in RTOSs' code and learn how to avoid them by implementing better code.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I3, relevance 4/5

It evaluates static analysis against the MISRA-C coding standard, a stated precedent relevant to language restrictions and safety-oriented verification.

Round 3 — rescoring of the included set

include, criterion I4, relevance 4/5

This provides MISRA-C restriction precedent, including the static-analysis approach, rules assessed, and observed compliance and violations across a 16-RTOS corpus. These findings could populate extraction fields on language restrictions, analysis technique, evaluation scale, and their rationale as a comparator for RQ1.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 41 OF 60

Safety-Oriented Evaluation of Embedded C Coding Methodologies for Automotive Software Using MISRA C and ISO 26262 Standards

2025 · 2025 International Conference on Electrical Systems & Automation (ICESA)

Keywords. MISRA C;ISO 26262;Embedded Systems;Automotive Software;Manual Coding;Model-Based Design; Measurement;Codes;ISO Standards;Manuals;Encoding;Software reliability;Safety;Standards;Automotive engineering;Software development management

Abstract. The growing complexity of automotive systems has led to an increased reliance on embedded software, emphasizing the need for safe, reliable, and maintainable coding practises. This paper presents a comparative evaluation of manual and model-based C code development approaches for automotive systems, focusing on MISRA C and ISO 26262 compliance. The analysis covers two automotive use cases—a Battery Management System and a Wiper Controller—evaluated across safety, maintainability, and traceability metrics. Results demonstrate that standards-compliant manual coding achieved 92.4% ASIL-B coverage with only 3 MISRA violations and 63.6% smaller memory footprint (28.7KB vs 78.9KB) compared to model-based approaches, while maintaining competitive fault detection rates (89.2% vs 95.8%). Although model-based methods reached slightly higher coverage (97.1% ASIL-B), they required 3.2x more code (1,742 vs 548 LOC). These findings establish standards-compliant manual coding as a practical alternative to commercial model-based tools while providing a comparative evaluation methodology for safety-critical automotive software development.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E3, relevance 2/5

Although it reports automotive C development under MISRA C, it does not address Python or dynamic-language restrictions, verification-tool soundness, or DO-178C objectives.

Round 2 — re-read of borderline records

include, criterion I4, relevance 4/5

The full paper could provide a MISRA C restriction precedent for RQ1, including the coding approach and its reported safety-related outcomes. Its two automotive use cases and compliance metrics could also populate the evaluation field.

Round 3 — rescoring of the included set

include, criterion I4, relevance 4/5

This could provide comparator evidence for RQ1 by describing MISRA C-compliant coding practice and its evaluation in safety-critical software. It may also populate the approach and evaluation fields, though the abstract does not identify specific restricted constructs or connect the results to DO-178C.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 42 OF 60

Specification-based testing for real-time avionic systems

1999 · IEE Colloquium on Applicable Modelling, Verification and Analysis Techniques for Real-Time Systems (Ref. No. 1999/006)

Abstract. The paper describes the early stages of work towards the development of specification based testing techniques for real time distributed systems. This work has taken place in the context of the aerospace industry, where the complexity and the criticality of avionic systems makes them a challenge to develop. This paper briefly presents the DoRIs/ADL notation and its proof theoretic semantics, and sketches the techniques used to represent test cases for single activities. We look forward to the issues dominating future work on the automatic generation of tests from DoRIs/ADL models.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I5, relevance 3/5

The paper describes a testing and test-generation approach for avionic systems, relevant as background on how verification evidence may be produced, but it does not address Python or specific DO-178C objectives.

Round 2 — re-read of borderline records

include, criterion I4, relevance 3/5

The paper describes an avionics testing approach, including a notation with proof-theoretic semantics and test-case representation, which can inform the verification-approach and applicable-activities fields for RQ3. It is indirect evidence: the abstract does not identify DO-178C objectives or a Python or comparator-language restriction.

Round 3 — rescoring of the included set

include, criterion I4, relevance 3/5

The paper could populate the verification approach and testing-method fields with specification-based testing for avionics; its relevance to the RQs is indirect because it does not address Python, a language subset, or specific DO-178C objectives.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 43 OF 60

EMI/EMC, lightning, radiation shielding verification approach for the Dragon COTS spacecraft: Part II

2011 · 2011 IEEE International Symposium on Electromagnetic Compatibility

Keywords. Lightning;Surges;Surge protection;Radar imaging;Aerospace electronics;Radio frequency

Abstract. Designing the Dragon COTS (Commercial Orbital Transfer System) for EMI/EMC interface compliance to the International Space Station (ISS) is a complex task. It involves designing Dragon for self-compatibility, compatibility with the Falcon 9 Launch Vehicle, designing to the LEO Space radiation environment, and designing for interface compatibility to the ISS mechanical, electrical and RF interfaces. This paper, Part II, describes the Space Exploration Technologies analysis and test verification methods used to achieve Dragon spacecraft self-compatibility, compatibility with the Eastern Launch Range RF requirements, and interface compatibility to the ISS requirements for EMC emissions and susceptibility.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E3, relevance 1/5

The verification concerns spacecraft electromagnetic compatibility, not Python, a comparable language, or software verification relevant to the review questions.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 44 OF 60

Test-based inference of polynomial loop-bound functions

2010 · Proceedings of the 8th International Conference on the Principles and Practice of Programming in Java

Keywords. termination, program verification, polynomial interpolation, loop-bound function inference, NCA-configuration, JML

Abstract. This paper presents an interpolation-based method of inferring arbitrary degree loop-bound functions for Java programs. Given a loop, by its "loop-bound function" we mean a function with the numeric program variables as its parameters, that is used to bound the number of loop-iterations. Using our analysis, loop-bound functions that are polynomials with natural, rational or real coefficients can be found.Analysis of loop bounds is important in several different areas, including worst-case execution time (WCET) and heap consumption analysis, optimising compilers and termination-analysis. While several other methods exist to infer numerical loop bounds, we know of no other research on the inference of non-linear loop-bound functions. Additionally, the inferred bounds are provable using external tools, e.g. KeY.To infer a loop-bound function for a given loop it is instrumented with a counter and executed on a well-chosen set of values of the numerical program variables. By well-chosen we mean that using these test values and the corresponding values of the counter, one can construct a unique interpolating polynomial. The uniqueness and the existence of the interpolating polynomial is guaranteed if the input values are in the so-called NCA-configuration, known from multivariate-polynomial interpolation theory. The constructed interpolating polynomial presumably bounds the dependency of the number of loop iterations on arbitrary values of the program variables. This hypothesis is verified by a third-party proof assistant.A prototype tool has been developed which implements this method. This prototype can infer piecewise polynomial loop-bound functions for a large class of loops in Java programs. Applicability of the prototype has been tested on a series of safety-critical case studies. For most of the loops in the case studies, loop-bound functions could be inferred (and verified using a proof assistant).

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I5, relevance 4/5

The paper demonstrates verification evidence for loop bounds in safety-critical Java case studies, making it relevant as a statically typed-language precedent for the review's DO-178C feasibility question.

Round 3 — rescoring of the included set

include, criterion I4, relevance 4/5

This Java analysis could populate the verification approach, property class (loop-iteration bounds), proof method, and evaluation fields, with safety-critical case studies providing comparator evidence for RQ3.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 45 OF 60

Real-time embedded software testing method based on extended finite state machine

2012 · Journal of Systems Engineering and Electronics

Keywords. real-time system;real-time embedded software;formal method;extended finite state machine (EFSM);testing sequence;test case; Real time systems;Embedded software;Time factors;Reliability;Software testing;Analytical models

Abstract. The reliability of real-time embedded software directly determines the reliability of the whole real-time embedded system, and the effective software testing is an important way to ensure software quality and reliability. Based on the analysis of the characteristics of real-time embedded software, the formal method is introduced into the real-time embedded software testing field and the real-time extended finite state machine (RT-EFSM) model is studied firstly. Then, the time zone division method of real-time embedded system is presented and the definition and description methods of time-constrained transition equivalence class (timeCTEC) are presented. Furthermore, the approaches of the testing sequence and test case generation are put forward. Finally, the proposed method is applied to a typical avionics realtime embedded software testing practice and the examples of the timeCTEC, testing sequences and test cases are given. With the analysis of the testing result, the application verification shows that the proposed method can effectively describe the real-time embedded software state transition characteristics and real-time requirements and play the advantages of the formal methods in accuracy, effectiveness and the automation supporting. Combined with the testing platform, the real-time, closed loop and automated simulation testing for real-time embedded software can be realized effectively.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I5, relevance 3/5

The avionics application and generated test sequences and cases provide relevant evidence about producing verification evidence for airborne software, though the record does not address Python or specific DO-178C objectives.

Round 3 — rescoring of the included set

include, criterion I4, relevance 3/5

The record describes an EFSM-based formal testing approach and its application to avionics software, which could populate the verification approach, test-case generation, and evaluation fields. It offers relevant testing context but does not identify a programming-language restriction or specific DO-178C objectives.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 46 OF 60

Wireless Attack Simulation of WiFi Threats: A Real-Time Scenario

2025 · 2025 Third International Conference on Industry 4.0 Technology (I4Tech)

Keywords. Wireless security;packet sniffing;deauthentication;MITM;evil twin;Python simulation;cyberattack modeling;network vulnerabilities; Wireless networks;Interactive systems;Real-time systems;Libraries;Fourth Industrial Revolution;Communication system security;Security;Cyberattack;Wireless fidelity;Resilience

Abstract. Wireless networks are more and more susceptible to numerous types of cyber attacks because they are open and shared. This work addresses the simulation in real time of four prominent wireless network attacks: Packet Sniffing, Deauthentication Attack, Man-in-the-Middle (MITM) Attack, and Evil Twin Attack. Utilizing Python and its networking libraries, we create an interactive system that simulates these attacks with randomized parameters to simulate real-world variation. The simulation provides dynamic feedback on attack success, detection probability, and possible data loss, enabling real-world analysis of each technique's effect. Our approach closes the gap between theoretical security models and real-world network contexts and offers an educational and research-based environment to study wireless vulnerability. Performance outcomes identify attack patterns under varied network settings and the capability of the framework to represent attacker strategy and network resilience.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 1/5

The work uses Python for wireless cyberattack simulation, but does not address language-runtime timing determinism, verification, language restriction, safety, or toolchain feasibility relevant to the review.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 47 OF 60

Optimizing and evaluating transient gradual typing

2019 · Proceedings of the 15th ACM SIGPLAN International Symposium on Dynamic Languages

Keywords. gradual typing / performance

Abstract. Gradual typing enables programmers to combine static and dynamic typing in the same language. However, ensuring sound interaction between the static and dynamic parts can incur runtime cost. In this paper, we analyze the performance of the transient design for gradual typing in Reticulated Python, a gradually typed variant of Python. This approach inserts lightweight checks throughout a program rather than installing proxies on higher order values. We show that, when using CPython as a host, performance decreases as programs evolve from dynamic to static types, up to a 6x slowdown compared to equivalent Python programs. To reduce this overhead, we design a static analysis and optimization that removes redundant runtime checks, employing a type inference algorithm that solves traditional subtyping constraints and also a new kind of check constraint. We evaluate the resulting performance and find that for many programs, the efficiency of partially typed programs is close to their untyped counterparts, removing most of the cost of transient checks. Finally, we measure the efficiency of Reticulated Python programs when running on PyPy, a tracing JIT. We find that combining PyPy with our type inference algorithm results in an average overhead of less than 1%.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I3, relevance 5/5

The paper develops and evaluates static analysis and type inference for a gradually typed Python variant, directly informing the review's questions about Python analysis and toolsets.

Round 3 — rescoring of the included set

include, criterion I3, relevance 5/5

This studies Reticulated Python's type inference and static analysis, providing evidence about the analysis technique and its evaluation that can inform comparison with the proposed Python toolset under RQ2. The focus is performance rather than safety verification, but the Python analysis technique itself is directly relevant.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 48 OF 60

Monotonic gradual typing in a common calculus

2018 · Companion Proceedings for the ISSTA/ECOOP 2018 Workshops

Abstract. Gradual typing refers to the notion that programs can be incrementally decorated with type annotations. Languages that support this approach to software development allow for programs being in various states of "typedness" on a scale ranging from entirely untyped to fully statically typed. Points in the middle of this typed-untyped scale create interactions between typed and untyped code, which is where gradual type systems differ. Each gradual type system comes with tradeoffs. Some systems provide strong guarantees at the expense of vastly degraded performance; others do not impact the running time of programs, but they do little to prevent type errors. This paper looks at an intriguing point in the landscape of these systems: the monotonic semantics. The monotonic semantics is designed to reduce the overhead of typed field access through a different enforcement mechanism compared to other gradual type systems. In our previous paper, [1], we described four semantics for gradual typing. This paper uses the framework of that companion paper to present and explore a formulation for the monotonic semantics. In comparison to the others, the monotonic semantics is designed to reduce the overhead of typed field access. We translate a common gradually typed source language to a common statically typed target language according to the monotonic semantics, allowing easy comparison to other gradual type systems in our framework.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I4, relevance 5/5

The paper formalizes a type-system technique for a gradually typed language, making it relevant to verification and static/type-checking approaches for dynamic languages.

Round 3 — rescoring of the included set

include, criterion I3, relevance 5/5

This formal gradual-typing approach could populate the extraction fields for verification technology, property guarantees, and translation from a gradually typed source language to a statically typed target. It is relevant as a type-system technique for dynamically typed languages, though it is not specifically safety-oriented.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 49 OF 60

Performance comparison of various garbage collectors on JVM for web services

2006 · 2006 International Symposium on Communications and Information Technologies, ISCIT

Abstract. The current demand of e-commerce has increased the requirement of efficiency on Java server-side applications. Hence, the continuous availability and the good response-time of the Java virtual machine are needed to satisfy the continuous incoming request from remote clients. This project studied the optimization of garbage collector in Java virtual machine particularly on 3-tier Java server-side application to increase the throughput of real times processing, full utilization of CPU time and more memory efficiency to handle more workloads. The characteristics and the architectures of five JikesRVM garbage collectors were studied. They are CopyMS, GenMS, SemiSpace, GenCopy and MarkSweep. The best performing garbage collectors was determined and the main causes of their overheads were identified. The benchmarking suite, SPECjbb2000 was used to emulate a 3-tier Java server-side application. The performance of the five different garbage collectors on SPECjbb2000 for large and small memory size was compared. In conclusion, CopyMS is found to have the best average throughput for large heap size whereas, GenMS has the best overall performance in memory constraint and can handle the most workloads. On the other hand, GenCopy and SemiSpace demonstrated higher efficiency in handling light workloads. The performance of garbage collectors is proportional to the heap size used. Memory fragmentation and long pause time are two main challenges to be overcome for increasing the application performance. Future works for optimizing garbage collectors are recommended at the end of this report

---

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I3, relevance 3/5

The study examines JVM garbage collectors and explicitly identifies pause time, providing relevant runtime-level timing background for RQ3, though it does not assess verification or certification objectives.

Round 2 — re-read of borderline records

exclude, criterion E4, relevance 2/5

This compares garbage collectors for web-service performance, but provides no verification, timing-predictability, safety, or certification evidence that would fill an extraction field for the RQs. Its runtime focus alone does not make a general-purpose performance comparison relevant.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 50 OF 60

Verifying semantic conformance of state machine-to-java code generators

2010 · Proceedings of the 13th International Conference on Model Driven Engineering Languages and Systems: Part I

Abstract. When applying model-driven engineering to safety-critical systems, the correctness of model transformations is crucial. In this paper, we investigate a novel approach to verifying the conformance to source language semantics of model-to-code transformations that uses annotations in the generated code. These annotations are inserted by the transformation and are used to guide a model checker to verify that the generated code satisfies the semantics of the source language - UML state machines in this paper. Verifying the generated output in this way is more efficient than formally verifying the transformation's definition. The verification is performed using Java Pathfinder (JPF) [1], a model checker for Java source code. The approach has been applied to verify three UML state machine to Java code generators: one developed by us and two commercial generators (Rhapsody and Visual Paradigm). We were able to detect non-conformance in both commercial tools, which failed some semantic properties extracted from the UML specification.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I5, relevance 4/5

It evaluates model checking of generated Java code to verify code-generator conformance, providing relevant precedent for verification evidence in a safety-critical toolchain.

Round 3 — rescoring of the included set

include, criterion I4, relevance 3/5

A full reading could populate the verification approach, annotations, properties checked, tool used (Java Pathfinder), and evaluation scale, including detected generator non-conformance. Its relevance is indirect: it concerns safety-critical model-driven development, but does not address Python, DO-178C objectives, or managed-runtime certification precedent directly.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 51 OF 60

On the Usefulness of Python Structural Pattern Matching: An Empirical Study

2024 · 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)

Keywords. python;structural pattern matching;user study; Codes;Reviews;Source coding;Linear regression;Programming;Particle measurements;Software

Abstract. As the important role of software in our modern world becomes more and more evident, the need for more complex data structures is increasing. Structural pattern matching has become an elegant technique for simplifying complex conditionallogic in the source code in modern programming languages. It provides an easy and concise way to destructure complex data and enables making decisions based on its structure, ultimately improving code readability. In the context of Python, a language renowned for its simplicity, structural pattern matching was a missing feature until October 2021. Finally, the feature is shipped in Python 3.10, allowing developers to exploit the potential of structural pattern matching. Instead of traditional conditional branching and endless type checking, structural pattern matching offers a more straightforward way to handle data. In this paper, we investigate the usefulness of this relatively new Python feature by involving 65 participants in a code review experiment. The participants (coming from diverse programming backgrounds) were presented with pairs of code snippets (one using structural pattern matching while the other using traditional conditional branching), each addressing the same task. They had to choose which code they preferred using a 4- point Likert scale based on three criteria: readability, modifiability, and personal preference. In the vast majority of cases, developers preferred code with structural pattern matching, but there were certain contexts in which a significant proportion of developers preferred the original code.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 1/5

Although the study concerns a Python language feature, it evaluates developer preferences rather than safety, verification, language restriction, or toolchain feasibility.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 52 OF 60

UCaG: An Automatic C Code Generator for AADL Based upon DeltaOS

2008 · 2008 International Conference on Advanced Computer Theory and Engineering

Keywords. AADL;Code generation;Delta OS; Yarn;Operating systems;Embedded system;Automotive engineering;Aerospace electronics;Real time systems;Open source software;Embedded computing;Physics computing;Application software

Abstract. Architecture analysis and design language (AADL), which is a modeling language standard brought forward by the Society of Automotive Engineers and Avionics Systems Division, is becoming a hot research topic for many organizations and institutes in the world. Though there are some tools could generate C code or Java code from a subset of AADL models as application program directly, it could not be applied in complex application without real-time embedded operating system. Based upon the hard-time operating system Delta OS, which is designed and developed by Coretek Systems Inc., this paper presents the mapping rules between the AADL elements and the elements in C language, designs an automatic C code generator and integrates it into OSATE which is an open-source tool developed by SEI, Carnegie Mellon University, and then, we use a case developed by Lambda tool that is the development and simulation environment for Delta OS to show the correctness of UCaG. Finally, we give the related work and future research directions.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 2/5

The paper concerns AADL-to-C code generation for an embedded operating system, but provides no evidence about language verification, safety-oriented restrictions, or the attainability of DO-178C objectives.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 53 OF 60

Efficient Algorithm for Jitter Minimization in Time-Triggered Periodic Mixed-Criticality Message Scheduling Problem

2016 · Proceedings of the 24th International Conference on Real-Time Networks and Systems

Keywords. Offline Scheduling, Mixed-Criticality, Message Scheduling

Abstract. The current research in real-time scheduling focuses mostly on the certification of functionalities with respect to safety requirements under conservative assumptions or to achieve efficient resource utilization but with optimistic assumptions. With growing system complexity, the safety certification is becoming hard, especially in event-triggered environments. In time-triggered environments, the network nodes are synchronized by clocks and follow a static schedule hence they are easily certifiable. However, the time-triggered paradigm has two disadvantages. The first one is its general non-flexibility (e.g. message retransmission, efficient resource usage) and the second one is the need for an efficient scheduling algorithm producing the schedule.In this paper, we propose a solution to both of these issues. To address the first disadvantage, we propose a method for non-preemptive message retransmission in time-triggered environments while preserving the efficient use of resources. Based on the message criticality we allow a certain number of retransmissions. The observed prolongation of the processing time of a highly critical message is compensated by skipping transmission of less critical messages. Static schedules then contain all alternatives caused by the retransmissions that can occur during a run time execution. Schedules conform with certification requirements imposed on the highly critical messages while preserving the efficient use of resources. To address the second disadvantage, we propose a novel heuristic scheduling algorithm with an unscheduling step for solving large instances of periodic message scheduling problem. The message periodicity is assumed to be a power of two and the objective is to minimize the maximal jitter. The efficiency of the approach is demonstrated on problem instances with up to 2000 messages.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 1/5

The work concerns periodic message scheduling and jitter minimization, not language-runtime timing, verification or language restriction, nor does it establish DO-178C objective attainability or how verification evidence is produced.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 54 OF 60

On design and formal verification of SNSP: a novel real-time communication protocol for safety-critical applications

2014 · Journal of Supercomputing

No abstract available — this record was judged on its title alone.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E3, relevance 1/5

The title indicates formal verification of a safety-critical protocol, but gives no evidence about Python or comparable dynamic languages or about the attainability of DO-178C objectives.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 55 OF 60

Real-time garbage collection for Java microprocessor

2008 · 2008 International Conference on Advanced Technologies for Communications

Keywords. Java;Microprocessors;Memory management;Real time systems;Yarn;Embedded system;Optimizing compilers;Communications technology;Software engineering;Educational institutions

Abstract. Problem in Java processor is small supported memory, and the device that using Java processor need to refresh (restart) memory manually in the ending of one thread. Automatic memory management of garbage collection greatly simplifies the development of large systems, but for using in mobile device, we need to control the memory size for it. However, garbage collection is used in that systems must running on real-time, it can be scheduled periodically in the same way as ordinary application threads. We provide an upper bound for the garbage collector period so that the application threads on mobile devices never run out of memory.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I3, relevance 3/5

The record provides relevant runtime-level evidence about periodic garbage collection in a real-time managed language, although it does not directly address verification or DO-178C objectives.

Round 2 — re-read of borderline records

include, criterion I3, relevance 4/5

This would populate the approach and runtime-behaviour fields with evidence about scheduling Java garbage collection to avoid memory exhaustion. Managed-runtime timing and memory behaviour is relevant precedent for RQ3.

Round 3 — rescoring of the included set

include, criterion I3, relevance 4/5

This provides managed-runtime precedent for RQ3: it would populate the verification approach and property fields with a scheduling bound for garbage collection intended to prevent application threads from running out of memory.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 56 OF 60

Scoped types for real-time Java

2004 · 25th IEEE International Real-Time Systems Symposium

Keywords. Java;Yarn;Runtime;Memory management;Virtual machining;Real time systems;Safety;Computer crashes;Proposals;Inspection

Abstract. A memory model based on scoped areas is one of the distinctive features of the Real-Time Specification for Java (RTSJ). Scoped types ensure timely reclamation of memory and predictable performance. The price to pay for these benefits is an unfamiliar programming model that, at the same time, is complex, requires checking all memory accesses, and rewards design-time errors with run-time crashes. We investigate an alternative approach, referred to as scoped types, that simplifies the task of managing memory in real-time codes. The key feature of our proposal is that the run-time partition of memory imposed by scoped areas is straightforwardly mirrored in the program text. Thus cursory inspection of a program reveals which objects inhabit the different scopes, significantly simplifying the task of understanding real-time Java programs. Moreover, we introduce a type system which ensures that no run-time errors due to memory access checks occur. Thus a RTSJ-compliant virtual machine does not require memory access checks. The contributions of this paper are the concept of scoped types, and a proof soundness of the type system. Experimental results are described in future work.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I5, relevance 4/5

Its scoped-type approach and soundness proof provide relevant safety-critical Java precedent on runtime memory safety and type-system verification, informing RQ3 even though it does not explicitly discuss DO-178C.

Round 3 — rescoring of the included set

include, criterion I4, relevance 4/5

This safety-oriented Java precedent can populate extraction fields on the scoped-memory restriction, its rationale, and the soundness claim for the type system. It is relevant to RQ3's assessment of managed-language memory behavior and verification, though it does not directly address DO-178C.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 57 OF 60

Edge-to-Cloud Iot Framework for Real-Time Urban Noise Mapping and AI-Driven Acoustic Classification

2026 · 2026 International Conference on System, Computation, Automation and Networking (ICSCAN)

Keywords. Urban Noise Mapping;IoT;MQTT;YAMNet;Deep Learning;InfluxDB;Mobile Crowdsensing;Smart City Analytics;Gemini AI;Acoustic Classification; Noise;Modeling;Printing;Artificial intelligence;Timing;Internet of Things;Smart phones;Real-time systems;Architecture;Computer architecture

Abstract. This paper presents the design and implementation of an end-to-end Internet of Things (IoT) framework for real-time urban noise mapping and intelligent acoustic classification. Traditional noise monitoring relies on expensive, static sound level meters that lack spatial granularity. To address this, we propose a mobile crowdsensing architecture where smartphones act as ubiquitous sensory nodes. Audio samples and GPS coordinates are captured via a cross-platform Flutter mobile application and transmitted via the lightweight MQTT protocol to a Dockerized backend. An automated Python-based consumer pipeline processes incoming streams, extracting core acoustic features including Root Mean Square (RMS) amplitude, Zero-Crossing Rate (ZCR), and Spectral Centroid. Audio is then passed through an AI-driven classification engine utilizing the YAMNet deep learning model to categorize sound sources into distinct classes such as Speech, Traffic, and Silence. Classified, geolocated data is stored in InfluxDB, a high-performance time-series database, and visualized on a live interactive web dashboard using Leaflet.js and Grafana. Additionally, Google Gemini AI generates automated acoustic forensics reports for government authorities. Experimental results show classification confidence exceeding 88% and total system latency under 600 ms.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 1/5

Although it uses Python and reports system latency, it does not address Python runtime timing behaviour, verification, safety, language restriction, or toolchain feasibility.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 58 OF 60

Formal Runtime Verification of Avionics Systems Using Level-D Flight Simulator Data and NASA/JPL's F Prime

2026 · Lecture Notes in Computer Science

No abstract available — this record was judged on its title alone.

What the automated screening decided

Round 1 — screening against the criteria

include, criterion I5, relevance 3/5

The title identifies a technical verification method applied to avionics, which may inform how verification evidence is produced for airborne software, although it does not establish a direct link to DO-178C objectives.

Round 2 — re-read of borderline records

include, criterion I4, relevance 3/5

A full reading could provide evidence about runtime-verification techniques, monitored property classes, and how verification is evaluated in an avionics context. This is relevant as context for verification activities, though the title does not establish a Python, DO-178C, or language-subsetting focus.

Round 3 — rescoring of the included set

include, criterion I4, relevance 3/5

A full reading could populate the verification approach, monitored property classes, and evaluation fields for an avionics context, providing indirect evidence relevant to RQ3. The title does not establish Python, DO-178C objectives, or certification evidence, so its contribution is contextual rather than direct.

Final decision: include

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 59 OF 60

Restart-based fault-tolerance: System design and schedulability analysis

2017 · 2017 IEEE 23rd International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)

Keywords. Safety;Software;Actuators;Ellipsoids;Switches;Timing

Abstract. Embedded systems in safety-critical environments are continuously required to deliver more performance and functionality, while expected to provide verified safety guarantees. Nonetheless, platform-wide software verification (required for safety) is often expensive. Therefore, design methods that enable utilization of components such as real-time operating systems (RTOS), without requiring their correctness to guarantee safety, is necessary. In this paper, we propose a design approach to deploy safe-by-design embedded systems. To attain this goal, we rely on a small core of verified software to handle faults in applications and RTOS and recover from them while ensuring that timing constraints of safety-critical tasks are always satisfied. Faults are detected by monitoring the application timing and fault-recovery is achieved via full platform restart and software reload, enabled by the short restart time of embedded systems. Schedulability analysis is used to ensure that the timing constraints of critical plant control tasks are always satisfied in spite of faults and consequent restarts. We derive schedulability results for four restart-tolerant task models. We use a simulator to evaluate and compare the performance of the considered scheduling models.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E3, relevance 1/5

The paper concerns system-level fault recovery and task schedulability, not Python or another language runtime, language restriction, verification toolchain, or attainable DO-178C objectives.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why:

RECORD 60 OF 60

Hybrid Real-Time Intelligent Code Translator

2025 · 2025 IEEE 9th International Conference on Information and Communication Technology (CICT)

Keywords. Neural Machine Translation (NMT);Abstract Syntactic Tree (AST);Identifier-Aware Semantic Model;Intermediate Representation (IR); Java;Translation;Codes;Trees (botanical);Semantics;Neural machine translation;Syntactics;Real-time systems;Software development management;Python

Abstract. In the modern world of software development, programmers often work across multiple programming languages making it difficult to recall language specific syntax and semantics. This paper proposes a real time intelligent code translation framework that allows developers to write syntactically correct code in the target language. For instances a developer is proficient in Python but is required to code in Java, this framework can assist by converting the Python code into Java. Thus, helping the user understand and learn the other language better. The framework combines neural machine translation (NMT) with identifier-aware semantic modeling, integrating Abstract Syntax Trees (ASTs) and Intermediate Representations (IRs) to preserve both syntactic accuracy and semantic fidelity. Unlike existing batch-mode translators, the proposed system performs incremental, low- latency translation suitable for IDE integration. Experimental evaluation on Python-Java and Python–C benchmarks show improved BLEU score, compilation success rate, and functional equivalence compared to state-of-the-art NMT models, with only a modest latency overhead. This work demonstrates the potential of hybrid AST+NMT approaches for intelligent, real-time multilingual code translation.

What the automated screening decided

Round 1 — screening against the criteria

exclude, criterion E4, relevance 1/5

The paper concerns Python-to-Java/C code translation and developer assistance, not verification, safety-oriented language restriction, runtime timing determinism, or DO-178C evidence.

Final decision: exclude

Do you agree with the final decision? Agree ☐ Disagree ☐

If you disagree, why: